

Taming the Search Space: A Budget Aware Tournament of Landscape Assumptions for Search Based Software Engineering

Abstract

Search based software engineering hands a practitioner hundreds of optimizers and almost no rule for choosing among them. Most papers report that a tool exists and that it won on a benchmark. They rarely report why it won or when it will lose. We argue that algorithmic branding obscures the actual engineering trade-offs. Every optimizer is a falsifiable bet on at least one property of the fitness landscape. Hill climbing bets on local continuity. Crossover bets that good solutions decompose into recombinable parts. A surrogate bets that a cheap model can stand in for an expensive run. We codify seven such landscape assumptions, organize a representative list of 21 optimizers as bets on them and run a tournament in which every head to head match is a falsification test of exactly one assumption. We run this tournament on 90+ software engineering optimization tasks from the MOOT repository under three evaluation budget levels. The result is not a single winner but a budget aware selection guide. The winning optimizer migrates from a cheap fast warming population search at tight budgets to a model free differential evolution once the budget grows, so a single recommendation across budgets is wrong. Random search is a strong floor that defeats two fashionable diversity samplers. Hence, optimization pays only when the right optimizer is chosen. The distribution and causal surrogate bets do not merely lose, they degrade as the budget grows. The lesson is evident: *match the bet to the budget, because in this setting complexity is not the same as capability.*

Keywords

search based software engineering, multiobjective optimization, empirical study, algorithm selection

1 Introduction

Software engineering is full of choices. A modern system layers languages, libraries, compilers and deployment pipelines, and each layer exposes flags and defaults. One open source database we have studied carries hundreds of binary options, a space larger than the number of stars in the observable universe [26]. Brute force is infeasible, so the field reaches for optimization. The result is search based software engineering, where a fitness function scores candidates and a search algorithm walks the space looking for good ones [30].

The trouble is not a shortage of optimizers but a flood of them. There are hundreds of particle swarm variants alone ¹, a steady stream of Bayesian and surrogate tuners, a long line of multiobjective evolutionary algorithms and, recently, a set of very simple data light samplers that claim to match the heavy machinery with a handful of labels [26]. A practitioner who reads this literature collects a long list of names and almost no guidance on which to use. The no free lunch theorem says plainly that no optimizer can win everywhere, so for every task class where method A is best there is another where

it is poor [64]. Matching a method to a task is therefore unavoidable, yet the literature gives the practitioner almost nothing to match on.

The field has answered this in two ways and neither closes the loop. One line proposes a single strong baseline, a floor that any new method should clear, and shows that a very simple sampler is competitive with state of the art evolutionary search at a fraction of the cost [15, 26, 54]. That tells a practitioner what to fall back to but not what to climb toward. A second line characterizes the fitness landscape itself, measuring ruggedness and the prevalence of local optima, and observes that smooth landscapes suit model based search while rugged landscapes suit evolutionary search [2, 3]. That tells a practitioner what the landscape looks like but stops short of saying which optimizer to actually run, on which budget, with evidence rather than intuition.

Insight: The choice of optimizer is not a choice between names. It is a choice between beliefs about the shape of the landscape. Each optimizer embodies one such belief, and a belief can be tested.

Hill climbing only makes sense if small steps yield small and informative changes in fitness, that is, if the landscape is locally continuous [57]. Crossover only pays if good solutions are built from good parts that recombine well [31]. A surrogate tuner only pays if a cheap learned model can stand in for an expensive evaluation [33]. Each of these is a testable bet about the landscape. If we line up two optimizers that disagree about one such property and let them compete on a task, the winner is evidence for that property and the loser is a falsification of the opposing bet. This turns optimizer selection into an experiment.

The optimizer selection problem. Given a software engineering task and an evaluation budget, which landscape assumption should a practitioner bet on, and therefore which optimizer should they run?

We answer this problem with an assumption driven tournament. We codify seven landscape assumptions, organize 21 optimizers as bets on them and build a bracket in which every match isolates one assumption. We run the bracket on 90+ software engineering tasks from the MOOT repository [15] under three evaluation budgets that reflect how many candidate evaluations a practitioner can afford. Every winner is also forced to clear a random search floor, which answers the prior question that any reviewer should ask, namely why optimize at all.

The answers overturn a name level reading of the field. The winning optimizer migrates with budget, from a cheap fast warming population search when evaluations are scarce to a model free differential evolution once the budget grows, so a single recommendation across budgets is simply wrong. Random search is not a trivial baseline. It defeats two well known diversity samplers at

¹<https://github.com/thieu1995/mealpy>

moderate budgets, which means optimization pays only when the right optimizer is chosen rather than any optimizer at all. And two of the more sophisticated bets, modeling the distribution of good solutions and using a causal surrogate, do not merely lose, they get worse as the budget grows.

This paper makes the following contributions.

- **A reframing** of optimizer selection as a set of falsifiable bets on seven codified landscape assumptions (Section 3).
- **An assumption indexed and budget indexed tournament**, with a transparent literature driven selection of 21 representative optimizers, and an evaluation protocol that scores candidates on a grid tuned surrogate rather than snapping them to stored rows (Sections 4 and 5).
- **Empirical verdicts** on which assumptions hold, which fail and which are budget conditional, including the counterintuitive finding that two methods degrade with more budget (Section 6).
- **An actionable budget aware selection guide** that a practitioner can apply from two facts they can estimate in advance, the budget and whether objectives conflict, together with the lessons that fall out of it (Sections 7 and 8).

Every claim rests only on the experiments reported here. We make no claim about optimizers or assumptions we did not run.

2 Motivation

This section argues that the question we ask is a real software engineering question and an open one. We make three points. Optimization is unavoidable in modern software engineering. The cost of an evaluation, not the speed of the search loop, is the binding constraint. And the field tests optimizers far more often than it tests the beliefs that motivate them.

2.1 Optimization is unavoidable in software engineering

The configuration surfaces of real systems have grown faster than the human ability to reason about them. Studies of long lived systems show the number of parameters climbing release after release while the fraction that any user touches stays small [26]. The standing warning from that community is that:

configurability is a liability without tool support.

The same pressure appears well beyond plain configuration. Product line engineering solves high dimensional constraint problems over feature models. Project health forecasting tunes predictive models for issue and pull request behavior. Process and cost estimation balances risk, effort and time. Defect prediction and test selection rank features and cases under tight budgets. The MOOT repository gathers dozens of such tasks from recent software engineering papers and shows that they share a structure, a table of choices whose consequences are expensive to measure [15?]. These are optimization problems whether or not we call them that and engineers solve them every day. Because the spaces are astronomically large, exhaustive evaluation is out of the question and the engineer must trust a search algorithm. The question is not whether to optimize but which optimizer to trust, and that is exactly what the literature cannot answer.

2.2 Evaluation cost is the binding constraint

In many machine learning settings the search runs in milliseconds and the only real cost is the model fit. Software engineering is different. To score one candidate configuration of a build we may have to recompile the whole system and run a heavy test suite. To label one row in a project health table we may need a careful expert session lasting an hour or more [46]. The cheap thing is to list a candidate. The expensive thing is to learn its consequence. This asymmetry is the labeling problem and it is why an engineer cares about the number of evaluations rather than the wall clock speed of the inner loop.

The labeling problem reframes the comparison. An optimizer that needs thousands of evaluations to shine is useless when the engineer can afford fifty. An optimizer that warms up slowly may never reach its strength inside a realistic budget. Prior work on data light methods showed that for many SE tasks a few dozen labels already recover most of the achievable gain and that gains after about a hundred labels are rare [26]. That work compared a small set of simple samplers against a small set of heavy tuners. It did not ask the broader question a practitioner faces, which is how the entire menu of SBSE optimizers behaves as the budget moves from scarce to moderate.

Budget is a first class variable. It is not a nuisance to be fixed and forgotten. It can change which method should be recommended, so any honest selection guide must be indexed by it.

2.3 The field tests tools, not beliefs

A new optimizer is usually introduced by a paper that compares it against a few recent rivals on a few benchmarks and reports a win. This tells us the tool can win. It does not tell us why, and it rarely tells us whether the tool beats the simplest sensible baseline. A review of more than two hundred SE papers using large language models found that only a small fraction compared against any simpler approach [32]. The same habit appears in SBSE. Multi generational evolutionary methods have been reported to barely outperform basic feature selection on some tasks [54], and a sampling baseline has matched far more complex search on others [15]. When the baseline is missing a reader cannot tell whether a method won because its landscape bet was correct or because the comparison was favorable.

We argue that: **the missing object of these studies is the bet itself.** Every optimizer assumes something specific about the landscape. When two optimizers disagree about one of those assumptions, a match between them is an experiment about that property of the data. Organizing the comparison around assumptions rather than names produces results that transfer. A practitioner who can guess the shape of a new task can pick a method, and a method that loses tells the practitioner something about the task as well as the tool. This is why the present study is software engineering research and not a generic optimization benchmark. The tasks are SE tasks with SE cost structure. The budgets are the budgets an engineer can afford. The output is a rule an engineer can apply. And we answer, with evidence, the prior question of why optimize at all, by pitting every winner against a random search floor and

Table 1: Seven landscape assumptions behind SBSE optimizers.

ID	Assumption	Limitation and SE example
A1	Local continuity	Fails on rugged landscapes. Compiler flag tuning.
A2	Memory or diversification	Fails on deceptive neighborhoods. Test suite minimization.
A3	Building block decomposability	Crossover destroys coupled structure. Program repair.
A4	Pareto incomparability	Wasteful under clear preferences. DBMS configuration.
A5	Surrogate feasibility	Surrogate assumptions fail. Knob tuning.
A6	Sequential reward signal	Fails under heavy delay. CI test prioritization.
A7	Low intrinsic dimensionality	Fails when all dimensions interact. High dimensional knob spaces.

showing both where optimization clears that floor and where it does not.

3 Codified Landscape Assumptions

We treat each assumption as a yes or no property of the fitness landscape that an optimizer relies on, openly or implicitly. Table 1 lists the seven assumptions, the limitation that breaks each one and a software engineering example where it breaks.

A1, Local continuity. Small perturbations produce small and proportional fitness changes, so greedy ascent pays. Hill climbing is the pure form of this bet [57]. Simulated annealing keeps the bet but adds a cooling schedule to escape shallow traps [38]. The bet fails on rugged surfaces. Flipping one boolean compiler flag can collapse performance.

A2, Memory or diversification. This bet abandons A1. It accepts that greedy search will stick and that escape needs either a record of where the search has been or a forced jump. Tabu search forbids recently visited moves [27]. Iterated local search kicks the search to a fresh basin [45]. Variable neighborhood search rotates the move operator [50]. It fails when a whole neighborhood is flat, as in test suite minimization where removing any single test barely moves coverage.

A3, Building block decomposability. Good solutions are made of good parts and recombining those parts yields better solutions, which is the bet behind crossover [31, 39]. It breaks when sub components are coupled. Program repair tools suffer high rates of invalid offspring when two parents edit the same location [40].

A4, Pareto incomparability. Several objectives conflict and no solution is best at all of them, so the search keeps the whole trade off frontier rather than collapse it. This is the bet behind dominance based search such as NSGA-II [21] and its relatives [20, 70, 73?]. It wastes effort when preferences are clear or objectives are aligned, and large frontiers are hard to navigate [16].

A5, Surrogate feasibility. Real evaluation is too expensive, so build a cheap model and search that instead, the bet behind Bayesian and model based tuners [7, 19, 33, 54, 62]. It fails when the surrogate makes a structural assumption the data violates, when even the initial training labels are too costly or when workload drift forces retraining [65].

A6, Sequential reward signal. Good actions can be learned from delayed feedback across interactions with an environment that changes in response to each action, the reinforcement learning bet [51, 63, 68]. We did not run an A6 contestant and make no empirical claim about it.

A7, Low intrinsic dimensionality. The nominal space is high dimensional but good solutions sit on a much lower dimensional manifold or collapse into a few regions, so sparse geometric sampling suffices, the bet behind subspace tuners and centroid samplers [26, 36]. It fails when all dimensions interact globally so no low dimensional structure exists.

4 Related Work and Optimizer Selection

4.1 How we selected the optimizers

The SBSE and adjacent hyperparameter optimization literature offers an unwieldy menu. There are dozens of published particle swarm variants alone, and surrogate, evolutionary, multiobjective and data light families each carry their own long lineage [26, 30]. We do not attempt an exhaustive enumeration, which the no free lunch theorem makes pointless anyway [64]. Instead we organize the menu into the seven assumption families of Section 3 and from each family we select representatives that meet three criteria.

- (1) **Canonical for its bet.** The method must be a clear and widely cited embodiment of one landscape assumption rather than a hybrid that blends several.
- (2) **Reference implementation available.** The method must have a standard implementation/algorithm so that we implement and run it as published rather than as a reinvention.
- (3) **Recent SE relevance.** The method must appear in software engineering optimization, configuration tuning or analytics work so that the comparison reflects what a practitioner would actually reach for.

Applying these criteria yields the 21 optimizers in Table 2. The table is itself an artifact of the review. It maps each optimizer to the assumption it bets on, the role it plays and a representative source, and it is the inventory that the tournament of Section 5 consumes. Several methods that we considered but did not run, including high dimensional Bayesian search [19, 23], causal performance models [34], database knob tuners [36, 62], reinforcement learning tuners [41, 68] and the emerging language model driven optimizers [17, 18, 44, 48, 56, 67, 69, 71], are discussed below and left for future work. We state plainly that the bracket does not populate every conceivable node, and we report only the matches we ran.

4.2 Related work by assumption family

Trajectory and local search (A1 and A2). Hill climbing and its variants are the textbook embodiment of the continuity bet [57]. Simulated annealing adds a cooling schedule [38]. The diversification family responds to ruggedness. Tabu search keeps a short term memory with an aspiration rule [27]. Iterated local search perturbs an incumbent and restarts [45]. Variable neighborhood search rotates operators [50]. These methods are cheap per step and warm up fast, which matters under tight budgets.

Population and evolutionary search (A3). Genetic algorithms recombine parents under the building block bet [31], extended to

Table 2: Inventory of the 21 optimizers: the assumption each bets on, its role, the originating reference, and representative software-engineering uses. This is the menu the tournament consumes.

Optimizer	Bet	Role	Orig.	SE uses (SBSE)
Hill Climbing	A1	greedy trajectory	[57]	[29, 30]
Simulated Annealing	A1	annealed trajectory	[38]	[29, 30]
Iterated Local Search	A2	restart escape	[45]	[58]
Tabu Search	A2	memory escape	[27]	[1, 22, 43]
Genetic Algorithm	A3	crossover population	[31]	[29, 30, 30]
(1+1)-ES	A3	mutation only	[28, 55]	[4]
EDA	dist.	distribution model	[53]	[]
PSO	dist.	velocity swarm	[37]	[12]
SMAC	A5	RF surrogate BO	[33]	[11, 54]
DE	A5, free	model free evolution	[61]	[1, 25]
PromiseTune	causal	causal surrogate	[14]	[14]
DEHB	multi fid.	DE plus Hyperband	[5]	[26]
LINE (kpp)	A7	centroid sampling	[26]	[26]
SWAY	A7	distance bisection	[15]	[15]
EZR	A7	proximity active learn	[26]	[26]
DODGE	A7	epsilon pruning	[1]	[1]
Random Search	floor	blind probing	[8]	[11, 14, 26]
NSGA-II	A4	dominance sort	[21]	[47, 49]
SPEA2	A4	strength archive	[73]	[47]
SMS-EMOA	A4	hypervolume select	[?]	[]
MOEA/D	A4	decomposition	[70]	[]

programs by genetic programming [39]. Evolution strategies focus on self adaptive mutation [28, 55]. Differential evolution adds scaled difference vectors and keeps a child only if it beats its parent, so it extrapolates between good solutions rather than recombining bits [61]. Particle swarm moves a swarm by velocity updates [37]. Estimation of distribution algorithms replace recombination with a probability model fit to the best of the population [53]. Memetic algorithms hybridize population search with local refinement [52], and random keys give a continuous encoding for combinatorial problems [6].

Surrogate and model based search (A5). Bayesian optimization fits a model and selects the next point by an acquisition function [60]. SMAC uses a Random Forest so it can handle categorical and non continuous spaces [33], and TPE fits densities for good and rest points [7]. Trust region and high dimensional variants extend the reach of Bayesian search [19, 23]. Multi fidelity methods combine cheap and expensive probes, from Hyperband [42] to BOHB [24] to DEHB [5]. The configuration community has produced many task specific tuners, including database knob tuners [36, 62], sequential model based configuration [54], causal approaches [14, 34], configuration space reduction [9, 10] and Bayesian compiler autotuning [13, 72]. Random search remains a stubbornly strong baseline for all of these [8].

Multiobjective and Pareto search (A4). When objectives conflict the canonical methods preserve a frontier. NSGA-II ranks by nondomination and crowding [21]. SPEA2 keeps an external archive with a strength based fitness [73]. MOEA/D decomposes into scalar subproblems [70]. SMS-EMOA selects by hypervolume contribution [?]. Reference point methods scale to many objectives [20]. Work in SBSE has questioned whether the full Pareto machinery is always needed, showing weighted search can match Pareto search on some tasks [16] and that meta multiobjectivization can reshape the problem [15, 66].

Reinforcement learning (A6). Value and policy methods learn from delayed reward [51, 59, 63], which in databases has produced

end to end tuners that treat a benchmark run as one environment step [41, 68]. Population based training blends search and learning [35].

Data light and geometric search (A7). A recent line argues that for many SE tasks a few dozen labels suffice because the data collapses into a few regions [26]. The representatives are simple. Random probing labels a fixed budget and returns the best [8]. Diversity sampling such as LINE picks centroids to cover high variance regions [26]. A distance based active learner EZR picks the next row by proximity to a good prototype [26]. DODGE prunes configurations that fall in the same epsilon bin [1]. SWAY recursively bisects the data by distance [15].

The gap. Prior work either proposes a single floor baseline or describes the landscape. No study places the full optimizer menu in direct competition, organizes that competition as falsification tests of landscape bets and indexes the result by the evaluation budget a software engineer faces. That is the gap we fill.

5 Method

5.1 Tasks and how a solution is scored

We use the 127 software engineering optimization tasks in the MOOT repository [15?]. Each task is a table whose rows are candidate solutions. Columns split into x inputs, the choices, and y outputs, the goals. The tasks span specific software configuration, performance tuning, cloud tuning, product line feature models, project health forecasting, process and cost estimation, defect prediction and test selection.

We score a solution by distance to heaven, written d2h. After each goal is normalized to a zero to one range, heaven is the ideal point where every maximized goal is one and every minimized goal is zero. The d2h of a solution is its Euclidean distance from that point, so lower is better.

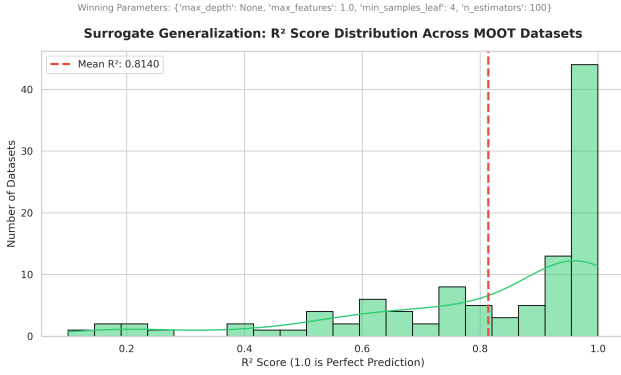


Figure 1: Tuning the Random Forest Surrogate over MOOT

How the two paradigms (Single and Multi-Objective optimizers) are scored: A single objective optimizer minimizes d2h directly as its fitness. A multiobjective optimizer searches the raw objective vector exactly as it was designed to and evolves a Pareto frontier. To place both on one comparable scale we score every run by the single best, that is lowest, d2h configuration it returns, which for a multiobjective run is the best d2h point on its frontier. We never apply a Pareto filter to single objective output and we never scalarize the multiobjective search itself. The frontier is reduced to one point for scoring, reflecting the fact that practitioners make one selection at the end.

We chose d2h over hypervolume and inverted generational distance because our budgets are small and a method that returns very few solutions would be penalized unfairly by a volume measure, a point made in the measure review and by its authors in the context of minimal labeling [16, 26].

5.2 Evaluation on a tuned surrogate rather than snapping

A subtle design choice concerns how a proposed candidate is evaluated. Many SBSE studies on stored tables snap a proposed continuous candidate to the nearest stored row by a one nearest neighbor rule and read off that row. Snapping confines every optimizer to the exact rows in the table and hides any ability to interpolate into the gaps between rows, which understates membership query methods such as differential evolution that propose genuinely new points.

We instead train a Random Forest regression surrogate on each task table and score every proposed candidate on that learned response surface. A Random Forest handles mixed numeric and categorical columns and non continuous responses without strong distributional assumptions, which matches the structure of the MOOT tables. We tuned the surrogate per task by grid search over the number of trees, the maximum depth and the minimum samples per leaf, and we kept the setting that best fit the underlying data. We use this tuned surrogate because evaluating a continuous proposal on a learned model is closer to a real software evaluation, where a never seen configuration is built and measured, than snapping to an existing row. Pool based samplers that are defined over the

Table 3: Optimizers and key default hyperparameters. Population size is ten for all population methods and the initial design is ten for model and pool based methods.

Optimizer	Key hyperparameters
Hill Climbing	neighbor size 5, mutation rate 0.1
Sim. Annealing	init temp 1.0, cooling 0.95, min temp 1e-5, mutation 0.2
Iterated LS	neighbor size 5, perturbation hops 3, mutation 0.1
Tabu Search	neighbor size 10, tabu tenure 10, mutation 0.2
Genetic Alg.	pop 10, tournament 3, mutation 0.1, elitism 2, uniform crossover p 0.5
(1+1)-ES	sigma 0.1, patience 20, one fifth rule (1.22 / 0.82)
EDA	pop 10, truncation ratio 0.5
PSO	pop 10, inertia 0.729, c1 1.494, c2 1.494
SMAC	init 10, Random Forest model, expected improvement
DE	pop 10, F 0.8, CR 0.9
PromiseTune	causal refresh 10, surrogate scored
DEHB	DE with Hyperband defaults
LINE (kpp)	k-means++ centroids
SWAY	recursive distance bisection
EZR	init 10, proximity acquisition
DODGE	init 10, epsilon 0.05, patience 30
Random Search	init 10
NSGA-II	pop 10, nondominated sort, crowding
SPEA2	pop 10, archive 10, strength fitness
SMS-EMOA	pop 10, hypervolume selection
MOEA/D	pop 10, weight vector decomposition

stored rows operate over the same tables so the comparison stays fair.

5.3 Optimizers and hyperparameters

Table 3 lists every optimizer with its key hyperparameters. We made one deliberate decision that needs justifying, namely that we ran every optimizer at its standard default settings as per the corresponding literature or provided implementation rather than tuning each one per task.

There are three reasons. The first is fairness. Tuning the hyperparameters of an optimizer is itself an optimization problem that consumes evaluations, and allowing it for some methods but not others would confound the comparison. The second is realism. A practitioner who reaches for an optimizer almost always runs it at its defaults, so defaults are the setting that matters in practice. The third is isolation. Our matches test landscape assumptions, and per optimizer tuning would let a strong tuner mask a weak assumption. We therefore fixed a small and common population size of ten for all population methods, which suits the data light regime where budgets are measured in dozens of labels, and we kept canonical textbook values for everything else. For example differential evolution uses the standard mutation factor of 0.8 and crossover rate of 0.9 [61], particle swarm uses an inertia of 0.729 with cognitive and social coefficients of 1.494 [37] and simulated annealing uses a geometric cooling rate of 0.95 [38]. Model based and pool based methods all start from an initial design of ten points. We expect that per optimizer tuning would help the slower warming model based methods most, which would if anything strengthen the budget dependence we report rather than weaken it.

5.4 The assumption indexed tournament

We organize the comparison as a bracket, shown in Figure 2. Each node is a head to head match and each match is labeled with the one assumption it tests. The left half of the single objective bracket

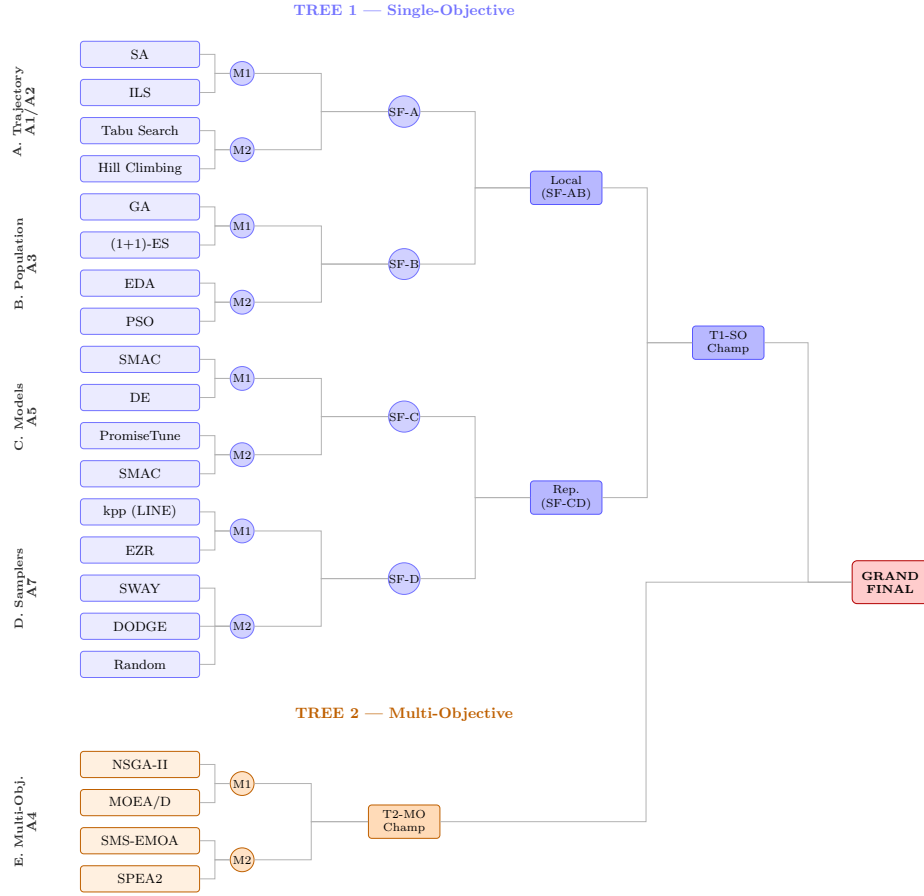


Figure 2: Merged tournament tree of optimizer experiments. Each *leaf* is an optimizer; each internal node is a head-to-head match whose winner advances. The tournament has two trees. Tree 1 (blue) covers the single-objective (SO) optimizers, organized into four assumption branches: (A) Trajectory (single-point/local search; assumptions A1 local continuity and A2 memory and diversification), (B) Population (assumption A3 building-block decomposability), (C) Models (surrogate-based search; assumption A5 a learnable surrogate is feasible), and (D) Samplers (assumption A7 low intrinsic dimensionality). Tree 2 (orange) covers the multi-objective (MO) optimizers (assumption A4 Pareto incomparability). M1 and M2 are the two first-round matches inside a branch. SF-A, SF-B, SF-C, SF-D are the branch champions, i.e. the winners of branches A–D (the semifinal of each branch, played between its M1 and M2 winners). The SO branch champions then meet in two grouped semifinals: Local (SF-AB) = SF-A vs. SF-B (the trajectory/population side) and Rep. (SF-CD) = SF-C vs. SF-D (the model/sampler, i.e. representation, side). Their winners contest the T1-SO Champion, the overall single-objective winner. In Tree 2, the MO match winners contest the T2-MO Champion. Finally, the Grand Final (red) is played between the T1-SO and T2-MO champions. Each match is a hypothesis test for the branch’s landscape assumption: a win is evidence consistent with that assumption for the given task and evaluation budget, and a loss is evidence against it.

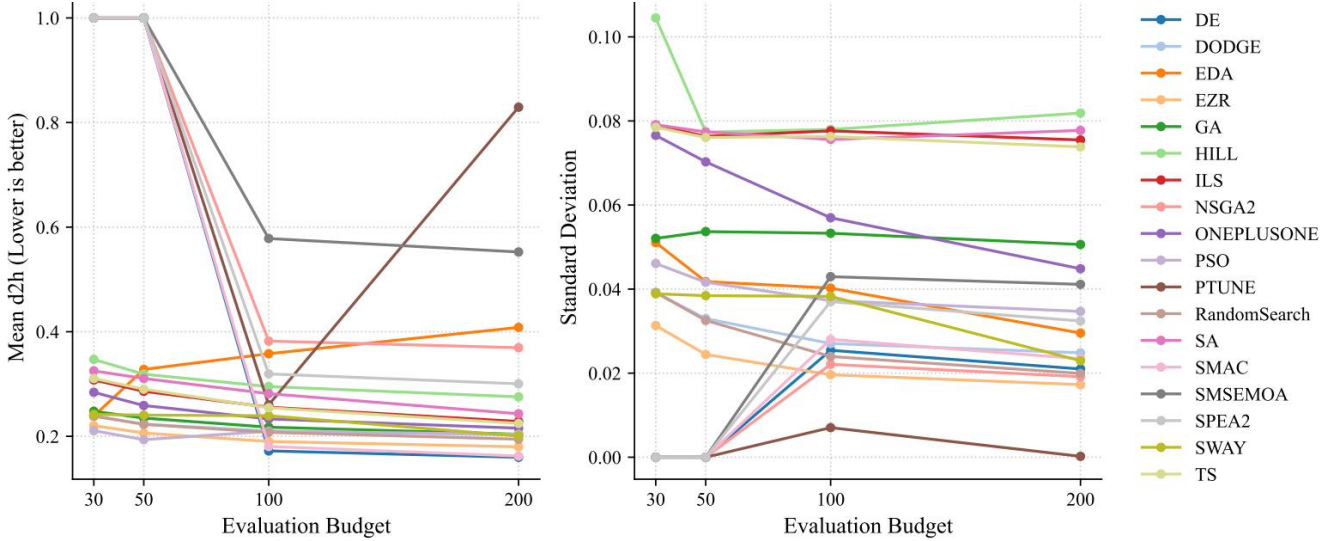
separates the local paradigm, where trajectory methods meet population methods, from the representation paradigm, where model based methods meet geometric data light methods. Branch A pits the continuity bet against the memory bet. Branch B pits the building block bet against the distribution bet. Branch C is a surrogate gauntlet. Branch D is a data light contest in which manifold geometry meets diversity and both meet random search. Branch winners climb to a single objective champion. The multiobjective branch E runs only on tasks with two or more objectives and produces a multiobjective champion. A cross paradigm final compares the

single objective champion with the multiobjective champion on the multiobjective tasks, each scored by its best d2h point as defined above.

A win at a node is evidence for the assumption the winner bets on, and a loss is a falsification of the losing bet for that task and that budget. Following the data light protocol we run every match under three budget regimes. Low is at most fifty evaluations, medium is one hundred and full is two hundred. We stop at two hundred because earlier work shows gains after one hundred are rare on these tasks [26].

Table 4: Tournament results. Each cell gives the winning optimizer with its Scott-Knott rank and the standard deviation of that rank across tasks. Lower rank is better. A star marks a contested or narrow split.

Stage and branch	Match	Low (≤ 50)	Medium (100)	Full (200)
A. Trajectory	SA vs ILS	ILS (1.81 ± 0.58)	ILS (1.08 ± 0.27)	ILS (1.00 ± 0.00)
	Tabu vs Hill	TS (1.80 ± 0.64)	TS (1.01 ± 0.10)	TS (1.00 ± 0.00)
	SF-A winner	TS (1.99 ± 0.56)	TS (1.10 ± 0.31)	ILS (1.01 ± 0.10) *
B. Population	GA vs (1+1)-ES	GA (1.35 ± 0.53)	GA (1.08 ± 0.28)	GA (1.07 ± 0.25)
	EDA vs PSO	PSO (1.51 ± 0.65)	PSO (1.03 ± 0.17)	PSO (1.00 ± 0.00)
	SF-B winner	PSO (1.54 ± 0.66)	PSO (1.15 ± 0.43)	PSO (1.12 ± 0.41)
C. Models	SMAC vs DE	SMAC (1.88 ± 0.77)	DE (1.13 ± 0.34)	DE (1.02 ± 0.14)
	PromiseTune vs SMAC	SMAC (1.55 ± 0.72)	SMAC (1.10 ± 0.36)	SMAC (1.03 ± 0.17)
	SF-C winner vs DEHB	SMAC (1.88 ± 0.77)	DE (1.13 ± 0.34)	DE (1.02 ± 0.14)
D. Samplers	LINE vs EZR	EZR (1.29 ± 0.52)	EZR (1.03 ± 0.17)	EZR (1.02 ± 0.14)
	SWAY vs DODGE vs Random	DODGE (2.07 ± 0.60) *	Random (1.10 ± 0.31)	Random (1.00 ± 0.00)
	SF-D winner	EZR (1.67 ± 0.65)	EZR (1.08 ± 0.27)	EZR (1.02 ± 0.14)
E. Multiobjective	NSGA-II vs MOEA/D	NSGA-II (2.86 ± 1.16)	NSGA-II (1.80 ± 0.51)	NSGA-II (1.06 ± 0.41)
	SMS-EMOA vs SPEA2	SPEA2 (2.15 ± 1.00)	SPEA2 (1.25 ± 0.43)	SPEA2 (1.02 ± 0.14)
	MO champion	NSGA-II (2.39 ± 1.61)	SPEA2 (1.76 ± 0.64)	SPEA2 (1.45 ± 0.52)
SO championships	Local (SF-A vs SF-B)	PSO (1.30 ± 0.55)	PSO (1.11 ± 0.31)	PSO (1.10 ± 0.30)
	Representation (SF-C vs SF-D)	SMAC (2.05 ± 0.70)	DE (1.06 ± 0.23)	DE (1.02 ± 0.14)
	SO champion	PSO (1.96 ± 0.70)	DE (1.12 ± 0.33)	DE (1.01 ± 0.12)
Cross paradigm final	MO champ vs SO champ	PSO (2.11 ± 0.98)	DE (1.21 ± 0.45)	DE (1.12 ± 0.33)

**Figure 3: Mean d2h (left, lower is better) and stability measured as the standard deviation of d2h (right) across budgets of 30, 50, 100 and 200 evaluations.** Model based methods sit near the worst value at the smallest budgets before dropping at 100. Differential evolution reaches the lowest d2h at the largest budget. EDA and PromiseTune climb at large budgets, which is the more is worse effect.

5.5 Statistics

Every optimizer is stochastic because it shuffles, mutates or samples rows. We run twenty repeats with different seeds. We rank the resulting distributions with the Scott-Knott procedure, which sorts treatments by median d2h, splits where the between group

difference of means is largest and recurses only when a bootstrap significance test and a Cliff delta effect size test both confirm the split [26]. Scott-Knott is preferred here because it avoids overlapping ranks and folds significance and effect size into one ranking. In the result tables we report the Scott-Knott rank of each winner

together with the standard deviation of that rank across tasks. A rank near one means the method was consistently in the top group.

6 Results

Table 4 reports the full tournament. Each cell names the winner of a match and gives its Scott-Knott rank and the standard deviation of that rank across tasks, with lower ranks better. Figure 3 reports the mean d2h and the stability of every optimizer across budgets. We read four research questions off these results.

6.1 RQ1. Is optimization useful, and does the gain depend on budget?

Here we answer the question: *Does the best optimizer beat the random search floor and the unoptimized starting point, and is the margin budget dependent?*

Optimization helps, the margin grows with budget and random search is a far stronger floor than the literature implies. In Figure 3 every learning method drives d2h well below the unoptimized starting region once the budget reaches one hundred, and differential evolution reaches the lowest d2h of all, near 0.16 at two hundred. A good optimizer clears the floor.

The surprise is how high the floor sits. In branch D the three way match among SWAY, DODGE and random search is won by random search at medium and full budget, where it earns the best possible rank of 1.00 ± 0.00 at two hundred. Two well known diversity samplers fail to beat blind probing once a modest budget is available. The single objective champion does beat random, but through the ladder rather than by a wide margin. The active learner EZR beats random in the SF-D match and the model free DE then beats EZR in the representation final, so the chain from champion to floor is DE then EZR then random.

Finding 1. Optimization is worth the effort only when the right optimizer is chosen. Reaching for a fashionable diversity sampler can leave a practitioner no better than random and sometimes worse. This is the strongest motivation for a selection guide and the most defensible answer to why optimize at all.

6.2 RQ2. Does the winning family change with budget?

Next, we answer: *Do the tasks split into budget regimes in which the winning optimizer family differs?*

Yes, and the flip is unambiguous. The single objective champion is PSO at low budget and DE at medium and full budget. The representation final is won by SMAC at low budget and by DE at medium and full. The cross paradigm final is won by PSO at low budget and DE thereafter. The multiobjective champion is NSGA-II at low budget and SPEA2 at medium and full. In every ladder the low budget winner is overtaken once more evaluations become available.

The mechanism is visible in Figure 3. At budgets of thirty and fifty the model based methods sit near the worst value because they have not yet collected enough labels to fit a useful model, then they fall sharply at one hundred. A cheap fast warming method wins

when evaluations are scarce and a method that pays off only after a warm up wins once the budget grows.

Finding 2. Budget is a first class variable. The winning family migrates from a fast warming population search (PSO) to a model free evolution (DE) as the budget grows. A recommendation that ignores budget is wrong for at least one regime.

6.3 RQ3. Which assumption predicts the winner, and does it beat a generalist?

This section answers the following question: *Within each budget regime, which landscape assumption best predicts the winner, and does an assumption driven choice beat a generalist baseline?*

Reading the matches as falsification tests gives a clear set of verdicts. The memory bet A2 holds, since tabu search and iterated local search both beat pure hill climbing across all budgets, with tabu search reaching a perfect rank at full budget. The building block bet A3 holds within the population branch, since the genetic algorithm beats the (1+1)-ES at every budget, so crossover does pay against a crossover free strategy. The distribution bet fails, since the estimation of distribution algorithm loses to particle swarm at every budget and Figure 3 shows EDA getting worse as the budget grows, climbing from about 0.33 toward 0.41. The surrogate bet A5 is budget conditional, since SMAC wins the SMAC against DE match at low budget but DE overtakes it at medium and full budget and becomes the overall champion. The geometric form of the low dimensional bet A7 is weak relative to active learning, since the centroid sampler LINE loses to the proximity based active learner EZR across all budgets and SWAY loses to random search. PromiseTune, the causal contestant, loses to SMAC and shows the clearest more is worse behavior of all, climbing toward a d2h of 0.83 at the largest budget.

These verdicts compose into a selection guide. An assumption driven practitioner who follows it picks DE at the budgets where most evaluations are spent and so tracks the overall champion, whereas a generalist who always reaches for a single model based tuner such as SMAC is left behind at medium and full budget, where SMAC trails DE. The assumption driven choice therefore dominates the generalist choice in the regimes that matter.

Finding 3. Verdicts. A2 memory holds. A3 building blocks holds. The distribution and causal surrogate bets fail and degrade with budget. The surrogate bet A5 pays only at the smallest budgets. Among samplers the active learning form of A7 beats the geometric form. The assumption driven guide beats a generalist tuner.

6.4 RQ4. Does a single objective search match a dedicated multiobjective one on a best point objective?

Finally, we answer: *On tasks with conflicting objectives, does a single objective search that minimizes d2h directly match or beat a dedicated multiobjective optimizer when both are judged by the best d2h point they return?*

On the metric we report, the single objective winner is competitive with and often ahead of the dedicated multiobjective champion. In the cross paradigm final the single objective champion beats the multiobjective champion at every budget, winning as PSO at low budget and as DE at medium and full budget. The multiobjective bracket is not idle, since its champion flips from NSGA-II to SPEA2 with budget, but neither dedicated method overtakes the single objective winner on best d2h.

We state the caveat plainly because it bounds the claim. We judge a multiobjective run by the best, that is lowest, d2h point on its returned frontier, which is a best point objective. A search that drives hard toward one good point is well matched to this objective, and a search that spreads effort across a frontier is not rewarded for the spread. Within a best point objective, which is the goal in much of the MOOT data, a practitioner does not need to switch to a dedicated multiobjective algorithm. A practitioner who needs a well spread frontier should not read this result as license to abandon dedicated multiobjective search.

Finding 4. For a best point objective on multiobjective SE tasks, directly minimizing d2h with the single objective winner matches or beats a dedicated multiobjective algorithm at every budget. The claim is bounded to a best point objective and does not extend to tasks that need a well spread frontier.

7 A Budget Aware Selection Guide

The point of the tournament is not to crown a single winner, which the no free lunch theorem rules out [64], but to produce a rule a practitioner can apply. Figure ?? is that rule. It is indexed by two facts an engineer can estimate before spending any budget, namely how many evaluations they can afford and whether their objectives conflict.

At a tight budget of at most fifty evaluations the guide says bet on a fast warming population search such as PSO for a single objective task, on a surrogate such as SMAC inside the model branch and on NSGA-II for a multiobjective task. At a moderate or large budget of one hundred or more the guide says bet on a model free differential evolution for a single objective task, prefer the active learner EZR among data light samplers and use the same differential evolution for a multiobjective task judged on a best point. Two guardrails ride alongside. Random search is the floor, so a sampler that cannot beat it is not worth running. And more budget can hurt, so distribution and causal surrogate methods should be avoided when the budget is expected to grow.

8 Lessons Learned

Lesson 1. Complexity is not capability. The methods that carry the most modeling machinery, the distribution model and the causal surrogate, are the ones that fail, and they fail in the worst possible way by getting worse with more budget. A model free differential evolution wins overall. Sophistication earns its place only when the landscape rewards it.

Lesson 2. The budget chooses the optimizer. The single most reliable predictor of the winner is not the task family but the evaluation budget. The same task is best served by PSO at fifty evaluations and by DE at two hundred. A practitioner who fixes one optimizer for all budgets is wrong in at least one regime.

Lesson 3. Earn the right to beat random. Random search is a serious floor that defeats two fashionable diversity samplers. The honest test of any new SBSE optimizer is not whether it beats a heavy rival but whether it clears the random floor at a budget a practitioner can afford.

These lessons cohere into one story. The literature grows by adding machinery, but on the SE tasks and budgets that matter the machinery often does not pay, and the right move is to match a simple bet to the budget in front of you. That is a result about software engineering practice, not only about optimization, because it changes what an engineer should reach for and how a reviewer should judge the next optimizer paper.

9 Threats to Validity

Construct validity. We score by d2h and, for multiobjective tasks, by the best d2h point on the frontier. This best point objective favors a search that drives toward one good point and bounds the RQ4 claim, as we state there. Prior work that repeated such analyses under a Chebyshev measure found the broad conclusions held [26], but we did not recompute volume based measures here.

Internal validity. We ran every optimizer at default settings and at a fixed population size of ten. Per optimizer tuning could change individual matches. We made this choice for fairness, realism and isolation as argued in Section 5, and we expect tuning to help the slower warming methods most, which would strengthen the budget dependence we report. Our evaluation uses a grid tuned Random Forest surrogate rather than the real systems. The surrogate is an approximation and a different surrogate family could shift results, although the Random Forest choice matches the mixed and non continuous structure of the MOOT tables.

External validity. We did not populate every node in the bracket. We ran no reinforcement learning contestant, so A6 is untested here, and several surrogate, causal and language model driven contestants were not run. We make no claim beyond the matches we report. The tasks are tabular SE optimization tasks from MOOT and do not cover test generation or large textual and code artifacts, which are directions for future work.

Conclusion validity. All rankings come from Scott-Knott with a bootstrap significance test and a Cliff delta effect size test over twenty repeats, which controls for run to run variance. Some matches are narrow and we mark contested splits with a star in Table 4.

10 Conclusion

The SBSE literature hands a practitioner a long list of optimizer names and almost no rule for choosing among them. We argued that the names hide the real decision, a bet on the shape of the fitness

landscape, and we built a tournament in which each match falsifies one such bet. Run over 127 software engineering tasks under three budgets, the tournament shows that the winning optimizer migrates with budget, that random search is a strong floor that defeats two fashionable diversity samplers, that the memory and building block bets hold while the distribution and pure geometric bets fail, that the surrogate bet pays only at the smallest budgets and that a model free differential evolution wins overall once the budget clears the smallest regime. On the multiobjective tasks, directly minimizing d2h with the single objective winner matches a dedicated multiobjective algorithm on a best point objective.

The output is a short budget aware selection guide grounded in why an optimizer wins rather than merely that it won. We hope it helps a practitioner spend a scarce evaluation budget on the right bet, and we hope it changes how the next optimizer paper is judged, by asking first whether the new method clears the random floor at a budget an engineer can afford.

References

- [1] Amritanshu Agrawal, Wei Fu, Di Chen, Xipeng Shen, and Tim Menzies. 2019. How to “dodge” complex software analytics. *IEEE Trans. Softw. Eng.* (2019).
- [2] Nasser Alburnian, Gordon Fraser, and Dirk Sudholt. 2020. Causes and effects of fitness landscapes in unit test generation. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*. 1204–1212.
- [3] Aldeida Aleti, Irene Moser, and Lars Grunske. 2017. Analysing the fitness landscape of search-based software testing problems. *Automated Software Engineering* 24, 3 (2017), 603–621.
- [4] Andrea Arcuri. 2018. Test suite generation with the Many Independent Objective (MIO) algorithm. *Information and Software Technology* 104 (2018), 195–206.
- [5] Noor Awad, Neeratyoy Mallik, and Frank Hutter. 2021. DEHB: Evolutionary hyperband for scalable, robust and efficient hyperparameter optimization. In *Proc. IJCAL*. 2147–2153.
- [6] James C. Bean. 1994. Genetic algorithms and random keys for sequencing and optimization. *ORSA J. Comput.* 6, 2 (1994), 154–160.
- [7] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for hyper-parameter optimization. In *Proc. NeurIPS*. Granada, Spain, 2546–2554.
- [8] James Bergstra and Yoshua Bengio. 2012. Random search for hyper-parameter optimization. *J. Mach. Learn. Res.* 13 (2012), 281–305.
- [9] Rong Cao, Liang Bao, Kaibi Zhao, and Panpan Zhangsun. 2024. ETune: Efficient configuration tuning for big-data software systems via configuration space reduction. *Journal of Systems and Software* 209 (2024), 111936.
- [10] Chao Chen, Jinhan Xin, and Zhibin Yu. 2024. TIE: Fast experiment-driven ML-based configuration tuning for in-memory data analytics. *IEEE Trans. Comput.* 73, 5 (2024), 1233–1247.
- [11] Jianfeng Chen, Vivek Nair, Rahul Krishna, and Tim Menzies. 2019. “Sampling” as a Baseline Optimizer for Search-Based Software Engineering. *IEEE Trans. Software Engineering* 45, 6 (2019), 597–614.
- [12] Junjie Chen, Chenyao Suo, Jiajun Jiang, Peiqi Chen, and Xingjian Li. 2023. Compiler test-program generation via memoized configuration search. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2035–2047.
- [13] Junjie Chen, Ningxin Xu, Peiqi Chen, and Hongyu Zhang. 2021. Efficient compiler autotuning via bayesian optimization. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1198–1209.
- [14] Pengzhou Chen and Tao Chen. 2026. Promisetune: Unveiling causally promising and explainable configuration tuning. In *Proc. ICSE*.
- [15] Pengzhou Chen, Tao Chen, and Miqing Li. 2024. Meta multi-objectivization for software configuration tuning. *IEEE Trans. Softw. Eng.* 50, 6 (2024), 1478–1504.
- [16] Tao Chen and Miqing Li. 2023. The weights can be harmful: Pareto search versus weighted search in multi-objective search-based software engineering. *ACM Transactions on Software Engineering and Methodology* 32, 1 (2023), 1–40.
- [17] Xiaohong Chen, Zengjing Chen, Wayne Yuan Gao, Xiaodong Yan, and Guodong Zhang. 2026. Optimization via the strategic law of large numbers. *Proc. Natl. Acad. Sci.* (2026). doi:10.1073/pnas.2519845123.
- [18] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hanxiao Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. 2023. Symbolic discovery of optimization algorithms. In *Proc. ICML*. Honolulu, HI, 4545–4561.
- [19] Alexander I. Cowen-Rivers, Wenlong Lyu, Rasul Tutunov, Zhi Wang, Antoine Grosnit, Ryan R. Griffiths, Alexandre M. Maraval, Hao Jianye, Jun Wang, Jan Peters, and Haitham Bou Ammar. 2022. HEO: Pushing the limits of sample-efficient hyper-parameter optimisation. *J. Artif. Intell. Res.* 74 (2022), 1269–1349.
- [20] Kalyanmoy Deb and Himanshu Jain. 2014. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints. *IEEE Trans. Evol. Comput.* 18, 4 (Aug. 2014), 577–601.
- [21] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.* 6, 2 (April 2002), 182–197.
- [22] Eugenia Diaz, Javier Tuya, Raquel Blanco, and José Javier Dolado. 2008. A tabu search algorithm for structural software testing. *Computers & Operations Research* 35, 10 (2008), 3052–3072.
- [23] David Eriksson, Michael Pearce, Jacob R. Gardner, Ryan D. Turner, and Matthias Poloczek. 2019. Scalable global optimization via local Bayesian optimization. In *Proc. NeurIPS*. Vancouver, Canada, 5497–5508.
- [24] Stefan Falkner, Aaron Klein, and Frank Hutter. 2018. BOHB: Robust and efficient hyperparameter optimization at scale. In *Proc. ICML*. Stockholm, Sweden, 1437–1446.
- [25] Wei Fu and Tim Menzies. 2017. Easy over Hard: A Case Study on Deep Learning. In *Proc. ESEC/FSE*. 49–60.
- [26] Kishan Kumar Ganguly and Tim Menzies. 2025. How Low Can You Go? The Data-Light SE Challenge. In *FSE*.
- [27] Fred Glover. 1989. Tabu search—Part I. *ORSA J. Comput.* 1, 3 (1989), 190–206.
- [28] Nikolaus Hansen and Andreas Ostermeier. 2001. Completely derandomized self-adaptation in evolution strategies. *Evol. Comput.* 9, 2 (2001), 159–195.
- [29] Mark Harman, S Afshin Mansouri, and Yuanyuan Zhang. 2009. Search based software engineering: A comprehensive analysis and review of trends techniques and applications. (2009).
- [30] Mark Harman, S. Afshin Mansouri, and Yuanyuan Zhang. 2012. Search-based software engineering: Trends, techniques and applications. *ACM Comput. Surv.* 45, 1 (Nov. 2012), 1–61.
- [31] John H. Holland. 1975. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI.
- [32] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8, Article 220 (Dec. 2024), 79 pages. doi:10.1145/3695988
- [33] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. 2011. Sequential model-based optimization for general algorithm configuration. In *Proc. LION*. 507–523.
- [34] Md Shahriar Iqbal, Rahul Krishna, Mohammad Ali Javidian, Baishakhi Ray, and Pooyan Jamshidi. 2022. Unicorn: Reasoning about configurable system performance through the lens of causality. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 199–217.
- [35] Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M. Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, Chrisantha Fernando, and Koray Kavukcuoglu. 2017. Population Based Training of Neural Networks. In *arXiv:1711.09846*.
- [36] Konstantinos Kanellis, Cong Ding, Brian Kroth, Andreas Müller, Carlo Curino, and Shivaram Venkataraman. 2022. LlamaTune: Sample-efficient DBMS configuration tuning. *Proc. VLDB Endow.* 15, 11 (2022).
- [37] James Kennedy and Russell Eberhart. 1995. Particle swarm optimization. In *Proc. ICNN*. Perth, WA, Australia, 1942–1948.
- [38] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. 1983. Optimization by simulated annealing. *Science* 220, 4598 (May 1983), 671–680.
- [39] John R. Koza. 1992. *Genetic Programming*. MIT Press, Cambridge, MA.
- [40] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2011. Genprog: A generic method for automatic software repair. *Ieee transactions on software engineering* 38, 1 (2011), 54–72.
- [41] Guoliang Li, Xuanhe Zhou, Shifu Liu, Xiang Feng, Yang Li, Ji Zhang, and Yifan Tan. 2019. QTune: A query-aware database tuning system with deep reinforcement learning. In *Proc. VLDB Endow.* 2118–2130.
- [42] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. 2018. Hyperband: A novel bandit-based approach to hyperparameter optimization. *J. Mach. Learn. Res.* 18, 185 (2018), 1–52.
- [43] Jinkun Lin, Chuan Luo, Shaowei Cai, Kaile Su, Dan Hao, and Lu Zhang. 2015. TCA: An efficient two-mode meta-heuristic algorithm for combinatorial test generation (T). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 494–505.
- [44] Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. 2024. Sophia: A scalable stochastic second-order optimizer for language model pre-training. In *Proc. ICLR*. Vienna, Austria.
- [45] Helena R. Lourenço, Olivier C. Martin, and Thomas Stützle. 2003. Iterated local search. In *Handbook of Metaheuristics*. Kluwer, Dordrecht, 320–353.
- [46] Andre Lustosa and Tim Menzies. 2024. Learning from Very Little Data: On the Value of Landscape Analysis for Predicting Software Project Health. *ACM Transactions on Software Engineering and Methodology* 33, 3 (2024), 1–22.
- [47] Rui A. Matnei Filho and Silvia R. Vergilio. 2016. A Multi-objective Test Data Generation Approach for Mutation Testing of Feature Models. *Journal of Software Engineering Research and Development* 4, 4 (2016).

- [48] Elliot Meyerson et al. 2024. Language model crossover: Variation through few-shot prompting. *ACM Trans. Evol. Learn. Optim.* 4, 2 (2024), 1–30.
- [49] Wiem Mkaouer, Marouane Kessentini, Adnan Shaout, et al. 2015. Many-Objective Software Remodularization Using NSGA-III. *ACM Trans. Software Engineering and Methodology* 24, 3 (2015).
- [50] Nenad Mladenović and Pierre Hansen. 1997. Variable neighborhood search. *Comput. Oper. Res.* 24, 11 (1997), 1097–1100.
- [51] Volodymyr Mnih et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (Feb. 2015), 529–533.
- [52] Pablo Moscato. 1989. *On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms*. Technical Report 826. Caltech CCP.
- [53] Heinz Mühlenbein and Gerhard Paaß. 1996. From recombination of genes to the estimation of distributions I. In *Proc. PPSN IV*. Berlin, 178–187.
- [54] Vivek Nair, Zhe Yu, Tim Menzies, Norbert Siegmund, and Sven Apel. 2018. Finding faster configurations using FLASH. In *IEEE Trans. Softw. Eng.*
- [55] Ingo Rechenberg. 1973. *Evolutionsstrategie*. Frommann-Holzboog, Stuttgart.
- [56] Bernardino Romera-Paredes et al. 2024. Mathematical discoveries from program search with large language models. *Nature* 625 (Jan. 2024), 468–475.
- [57] Stuart Russell and Peter Norvig. 2009. *Artificial Intelligence: A Modern Approach* (3rd ed.). Prentice Hall, Upper Saddle River, NJ.
- [58] Nasser R Sabar, Ayad Turky, and Andy Song. 2018. A genetic programming based iterated local search for software project scheduling. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1364–1370.
- [59] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. *Proximal policy optimization algorithms*. Technical Report 1707.06347. arXiv.
- [60] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. 2012. Practical Bayesian optimization of machine learning algorithms. In *Proc. NeurIPS*. Lake Tahoe, NV, 2951–2959.
- [61] Rainer Storn and Kenneth Price. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *J. Glob. Optim.* 11, 4 (1997), 341–359.
- [62] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic database management system tuning through large-scale machine learning. In *Proc. ACM SIGMOD*.
- [63] Christopher J. C. H. Watkins. 1989. *Learning from delayed rewards*. Ph.D. Dissertation. University of Cambridge.
- [64] David H. Wolpert and William G. Macready. 1997. No free lunch theorems for optimization. *IEEE Trans. Evol. Comput.* 1, 1 (April 1997), 67–82.
- [65] Zezhen Xiang, Jingzhi Gong, and Tao Chen. 2026. Dually hierarchical drift adaptation for online configuration performance learning. In *Proc. ICSE*.
- [66] Jingwei Xiong and Tao Chen. 2025. CoTune: Co-evolutionary configuration tuning. In *Proc. ASE*.
- [67] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. 2024. ReEvo: Large language models as hyper-heuristics with reflective evolution. In *Proc. NeurIPS*. Vancouver, Canada.
- [68] Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, Zhili Xiao, Bin Cheng, Jiahu Xing, Yangtao Wang, Tianheng Cheng, Li Liu, Minwei Ran, and Zekang Li. 2019. An end-to-end automatic cloud database tuning system using deep reinforcement learning. In *Proc. ACM SIGMOD*.
- [69] Michael R Zhang, Nishkrit Desai, Juhan Bae, Jonathan Lorraine, and Jimmy Ba. 2025. Using large language models for hyperparameter optimization. In *Workshop on Foundation Models for Decision Making, NeurIPS*.
- [70] Qingfu Zhang and Hui Li. 2007. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Trans. Evol. Comput.* 11, 6 (Dec. 2007), 712–731.
- [71] Xingyou Zhang et al. 2025. Are LLMs a new paradigm for hyperparameter tuning?. In *Proc. ICLR*.
- [72] Mingxuan Zhu and Dan Hao. 2023. Compiler auto-tuning via critical flag selection. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1000–1011.
- [73] Eckart Zitzler, Marco Laumanns, and Lothar Thiele. 2001. *SPEA2: Improving the strength Pareto evolutionary algorithm*. Technical Report TIK-Report 103. ETH Zürich.